Kunjal Purohit

kunjalp126@gmail.com | LinkedIn | GitHub | Portfolio | San Francisco Bay Area

Education

University of California, Santa Cruz — Santa Cruz, CA

Bachelor of Science in Computer Science and Bachelor of Science in Computer Engineering — *Expected: June 2028*

- **GPA: 3.95/4.0**
- **Honors:** Dean's Honors List (3x); American High School Principal's Award in Technology (top 2 of 600+)
- Relevant Coursework: Data Structures & Algorithms, Computer Systems & C, Assembly Language, Discrete Mathematics, Probability & Statistics, Linear Algebra, Multivariable Calculus, Differential Equations

Skills

Languages: Python, TypeScript, JavaScript, C/C++, Java, Swift, SQL (PostgreSQL), HTML/CSS, Assembly
Frameworks & Libraries: Next.js, React, Node.js, FastAPI, Capacitor, Convex, Playwright, Tailwind CSS, OpenCV
Tools & Platforms: Supabase, Neo4j, Clerk, Plaid, Vercel, Codemagic, Unix/Linux
Embedded Systems: STM32, ESP32, CAN Bus, OneWire, Sensor Integration, Signal Filtering

Experience

Embedded Software Engineer Intern — October 2025 – Present

Formula Slug (FSAE) – Student-Built Electric Race Car Team — *Santa Cruz, CA*

- Contribute to firmware that monitors and protects an electric race car's **126 V** battery pack
- Built STM32 C/C++ firmware that turns the pack's **30 cell voltages and 60 temperatures** into CAN telemetry, including tray-temperature monitoring on 4 DS18B20 sensors that triggers **automatic shutdown** on faults
- Replaced a piecewise pedal-to-torque map with a smooth cubic-polynomial curve (41-point LUT) for a smoother throttle response, now the **default accelerator mapping** in VCU firmware
- Redesigned the CAN protocol for a new battery and chip architecture, **adopted team-wide**, and documented the pack's telemetry data (units, ranges, resolution) so teammates can decode it without reading the firmware

Web Development Lead — August 2026 – Present

Association for Computing Machinery (ACM) — *Santa Cruz, CA*

- Lead a **4-person team** rebuilding the website of UCSC's largest CS club in Next.js, TypeScript, and Tailwind
- Built the shared UI baseline, laid the member-login foundation, and redesigned the Events page for **15+ events**
- Wrote a UI handoff guide so teammates can build in their own areas without inventing new styles

Projects

UCSC Dining Tracker | App Store | *Next.js, TypeScript, Supabase, Python, Swift* — June 2026 – Present

- Shipped a web and iOS macro tracker that makes UCSC dining nutrition easy to find, reaching **80+ downloads**
- Indexed **233K+ menu records** across 14 dining halls with daily Python/Playwright scrapers on GitHub Actions (**100% success over 470+ runs**), stored with 90+ days of history in Supabase PostgreSQL with per-user RLS
- Automated iOS builds, code-signing, and App Store deployments with a Codemagic CI/CD pipeline that runs on every merge, producing 64 builds and **4 App Store releases** (v1.0–v1.3)

Dowser (Oil Well Detection) | Live App | *Next.js, Convex, Capacitor, ESP32, YOLO* — October 2026

- Built in 42 hours with a team of 4 at Cal Hacks 13.0 an app that flags candidate oil wells by matching well symbols on old USGS topo maps against **158,000 California state well records**
- Built the iOS app and ESP32 prototype, auto-confirming a simulated gas signal (8 readings $\geq$ 1000 in 10 s)

Heirloom (AI Multi-Generational Wealth Platform) | *Next.js, FastAPI, Neo4j, Plaid, Daytona* — October 2026

- **Won 1st place in the Daytona track** (20 teams, $250 prize) at SF Tech Week by building a multi-generational AI wealth platform in 12 hours with **3 AI agents**, a Neo4j knowledge graph, and Plaid
- Designed a pipeline of 3 AI agents (intake, analyst, advisor) over **8 deterministic checks** that flags conflicts between parents' finances and heirs' goals and routes urgent cases to the wealth manager
- Built two-layer data isolation so advisors see only their own firm's clients (RLS plus org scoping on **16 tables**)

Autonomous Vehicle (UCSD COSMOS) | *Python, OpenCV, ROS, LiDAR, GNSS, Jetson Nano* — July 2024

- Enabled **3 fully autonomous laps** of a 1/10-scale car on a 5-person team by writing Python drivers for a LiDAR and GNSS receiver and fusing both into one log time-aligned at **0.05 s** on a Jetson Nano
- Built occupancy-grid mapping, localization, and real-time OpenCV lane tracking and obstacle detection, validated at **3 levels**: simulated ground truth from a bicycle-model generator, an indoor hallway, and an outdoor courtyard